



Con uso de `palette()`

1. ¿Qué es `palette()`?

En PySide6, cada widget (botones, cajas de texto, ventanas...) tiene asociada una **paleta de colores**.

La paleta define qué colores utiliza ese widget en sus distintas partes: fondo, texto, botones, zona editable, texto seleccionado, etc.

Cuando se llama a:

```
1 palette = widget.palette()
```

se obtiene la paleta actual del widget.

Esta paleta puede modificarse para ajustar colores de forma dinámica y, más tarde, aplicarse de nuevo con:

```
1 widget.setPalette(palette)
```

Una forma sencilla de entenderlo:

la paleta es "la caja de colores" que Qt usa para pintar un widget.

2. ¿Para qué sirve `palette()`?

Resulta útil cuando se quiere cambiar la apariencia de un widget **como respuesta a lo que ocurre en el programa**.

Por ejemplo:

- Mostrar un fondo rojo en un `QLineEdit` cuando el texto no es válido.
- Cambiar el color de un botón cuando se encuentra deshabilitado.
- Indicar estados como "ok", "aviso" o "error" mediante colores.

Es una herramienta pensada para modificar colores **desde el código y durante la ejecución**.

3. Métodos más útiles de `QPalette`

`setColor(role, color)`

Modifica el color asociado a un rol concreto.

Ejemplos frecuentes:

```
1 paleta.setColor(QPalette.Base, QColor("white"))      # Fondo c  
2 paleta.setColor(QPalette.Text, QColor("black"))      # Texto r  
3 paleta.setColor(QPalette.Window, QColor("#F0F0F0"))  # Fondo c  
4 paleta.setColor(QPalette.Button, QColor("#DDDDDD"))  # Fondo c  
5 paleta.setColor(QPalette.ButtonText, QColor("blue")) # Texto c
```

Es el método más usado al trabajar con validaciones visuales.

`color(role)`

Devuelve el color actual asignado a un rol.

```
1 color_actual = paleta.color(QPalette.Base)
```

Sirve para consultar colores antes de modificarlos o para restaurarlos más tarde.

`setBrush(role, brush)`

Permite asignar una **brocha** en lugar de un color plano.

Una brocha puede representar un color, un degradado o un patrón.

Ejemplo:

```
1 from PySide6.QtGui import QBrush  
2 paleta.setBrush(QPalette.Base, QBrush(QColor("yellow")))
```

`brush(role)`

Devuelve la brocha asociada al rol indicado.

```
1 mi_brocha = paleta.brush(QPalette.Button)
```

4. Roles de color más utilizados

Qt organiza los colores en roles. Algunos de los más importantes son:

Rol	Qué representa
<code>QPalette.Base</code>	Fondo de la zona editable (por ejemplo en <code>QLineEdit</code>)
<code>QPalette.Text</code>	Color del texto
<code>QPalette.Window</code>	Fondo general de una ventana
<code>QPalette.WindowText</code>	Texto de la ventana
<code>QPalette.Button</code>	Fondo de los botones
<code>QPalette.ButtonText</code>	Texto de los botones
<code>QPalette.Highlight</code>	Fondo del texto seleccionado
<code>QPalette.HighlightedText</code>	Texto seleccionado

Estos roles permiten modificar áreas concretas del widget sin afectar al resto.

5. Cuándo usar `QPalette` y cuándo usar QSS

Cuándo usar `QPalette`

Es la mejor opción cuando se necesita cambiar colores de forma **dinámica**, es decir, mientras el programa está funcionando.

Situaciones típicas:

- Validaciones de campos (`QLineEdit` que cambia a rojo si hay un error).
- Cambiar colores en función de estados variables.
- Personalizaciones puntuales desde Python.

Se centra en **colores**. No permite bordes, márgenes, radios ni estilos complejos.

Cuándo usar QSS (Qt Style Sheets)

QSS funciona de manera similar a CSS y permite definir estilos más detallados:

- Bordes
- Colores
- Tamaños
- Padding y margen
- Radio de borde
- Estados como hover o focus

Se usa principalmente cuando se quiere crear una apariencia más cuidada o cuando se quiere aplicar un estilo uniforme a toda la interfaz.

Ejemplo:

```
1 line_edit.setStyleSheet("""
2     QLineEdit {
3         background-color: #FFCCCC;
4         border: 2px solid red;
5         border-radius: 5px;
6     }
7 """)
```

6. Comparación

Aspecto	QPalette	QSS
Cambios dinámicos según la lógica del programa	✓ Muy adecuado	Puede usarse, pero no es cómodo
Validación de campos	✓ La mejor opción	Posible, pero menos natural
Estilo general de la aplicación	Limitado	✓ Ideal
Bordes, radio, padding, tipografías	✗ No disponible	✓ Completo
Nivel de detalle del estilo	Básico	Avanzado
Modificación por widget	✓	✓
Modificación global	Poco práctico	✓ Muy sencillo

7. Idea final

- `palette()` → colores dinámicos controlados desde el código.
- **QSS** → apariencia global y detalles visuales avanzados al estilo CSS.

Ambas herramientas pueden coexistir en una misma aplicación según la necesidad del momento.

Ejemplo

```
1  import sys
2  from PySide6.QtWidgets import (
3      QApplication,
4      QMainWindow,
5      QWidget,
6      QVBoxLayout,
7      QLabel,
8      QLineEdit
9  )
10 from PySide6.QtGui import QPalette, QColor
11
12
13 class LineEditAvisoColores(QLineEdit):
14     """
15     QLineEdit que cambia de color según la longitud del texto:
16
17         - Vacío          -> blanco
18         - OK              -> verde suave
19         - Aviso           -> amarillo suave
20         - Error (muy largo) -> rojo suave
21     """
22
23     def __init__(self, parent=None):
24         super().__init__(parent)
25
26         # Valores usados internamente
27         self.__longitud_avisos = 10
28         self.__longitud_error = 20
29
30         # Conectamos la señal
31         self.textChanged.connect(self.__comprobar_longitud)
32
33         # Primera comprobación
34         self.__comprobar_longitud("")
35
36     def __comprobar_longitud(self, texto):
```


